

Randomized Testing

Warmup: Mutation Testing

```
static boolean isAdult(int age) {  
    return age >= 18;  
}
```

```
@Test void t1() { assertTrue(isAdult(30)); }
```

```
@Test void t2() { assertFalse(isAdult(5)); }
```

Under Mutations = { gte -> gt, gte -> lte, numeric_literal -> numeric_literal + 1,
return boolean_expr -> return true }

What is the mutation score?

Review: Defects4J

- Defects4j checkout

Announcements

HW3 due last night

HW4 Mutation testing released

- Due after Fall break

Outline

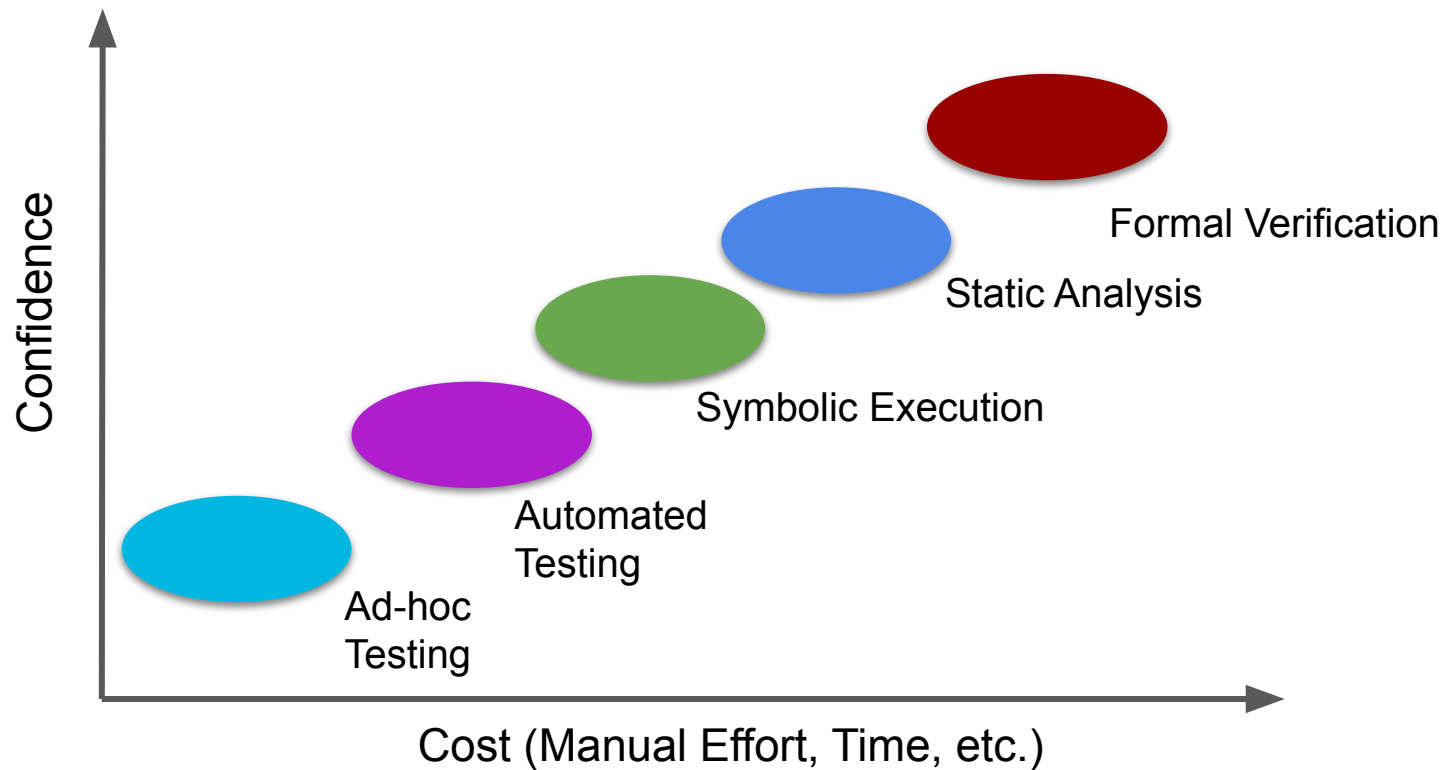
- Randomized Testing
- Randoop

What have we learned so far?

- Program analysis is impossible
- Program transformations
- How to evaluate a test suite
 - Coverage
 - Mutation testing
- Given a prefix, approaches to generate an assertion
- **Today we will learn how to generate the prefix**

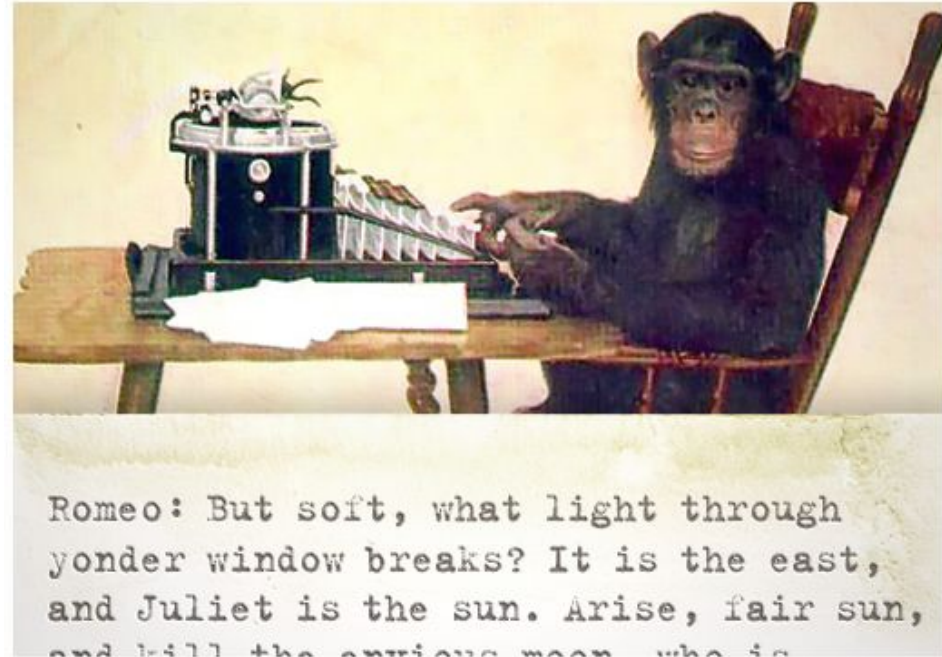
Automated Software Testing

Landscape of Program Analysis Techniques



The Infinite Monkey Theorem

“A monkey hitting keys at random on a typewriter keyboard will produce any given text, such as the complete works of Shakespeare, with probability approaching 1 as time increases.”



Randomized Testing

```
public static double divide(int x, int y) {  
    return x / y;  
}
```

```
class Number {  
    private double value;  
    public Number(double value) {  
        this.value = value;  
    }  
    public double getValue() {  
        return value;  
    }  
    public Number divide(Number other) {  
        return new Number(this.value / other.getValue());  
    }  
    public String toString() {  
        return Double.toString(value);  
    }  
}
```

Randomized Testing for OOP (Java)

Idea: a test can be built up iteratively by randomly selecting a method or constructor to invoke, using previously computed values as inputs.

Problem with uniform random testing: Creates too many **illegal** or **redundant** tests

Exercise

Value_pool = {-1, 0, 1, 2, 3, 4},

- Generate random sequences by “rolling a die” (rand int 1-6)
- Roll a die to pick a method: 1-4 => Fraction constructor; 5-6 times
- Roll for each argument. For an int, value_pool[num-1]. For a Fraction, pick one from the fractions you have already built

```
class Fraction {
    int num, den;
    Fraction(int n, int d) {
        if (d == 0) throw new IllegalArgumentException();
        num = n; den = d;
    }
    Fraction times(Fraction g) {
        return new Fraction(num * g.num, den * g.den);    // never reduces
    }
}
```

Random Testing: Pitfalls

1. Useful test

```
Set t = new HashSet();  
s.add("hi");  
assertTrue(s.equals(s));
```

2. Redundant test

```
Set t = new HashSet();  
s.add("hi");  
s.isEmpty();  
assertTrue(s.equals(s));
```

3. Useful test

```
Date d = new Date(2006, 2, 14);  
assertTrue(d.equals(d));
```

4. Illegal test

```
Date d = new Date(2006, 2, 14);  
d.setMonth(-1); // pre: argument >= 0  
assertTrue(d.equals(d));
```

5. Illegal test

```
Date d = new Date(2006, 2, 14);  
d.setMonth(-1);  
d.setDay(5);  
assertTrue(d.equals(d));
```

Randoop



Randoop

Automatic unit test generation for Java

To get around this, **Randomly** create new test **guided by feedback** from previously created tests

test == method sequence

Idea:

- Build new sequences incrementally, extending past sequences
- As soon as a sequence is created, execute it
- Use execution results to guide test generation towards sequences that create new object states

Randoop

Input:

- classes under test
- time limit
- set of contracts

e.g. “o.hashCode() throws
no exception”

e.g. “o.equals(o) == true”

Output:

- contract-violating test cases

```
LinkedList l1 = new LinkedList();  
Object o1 = new Object();  
l1.addFirst(o1);  
TreeSet t1 = new TreeSet(l1);  
Set s1 = Collections.unmodifiableSet(t1);  
assert(s1.equals(s1));
```

No contract violated up to here

fails when executed

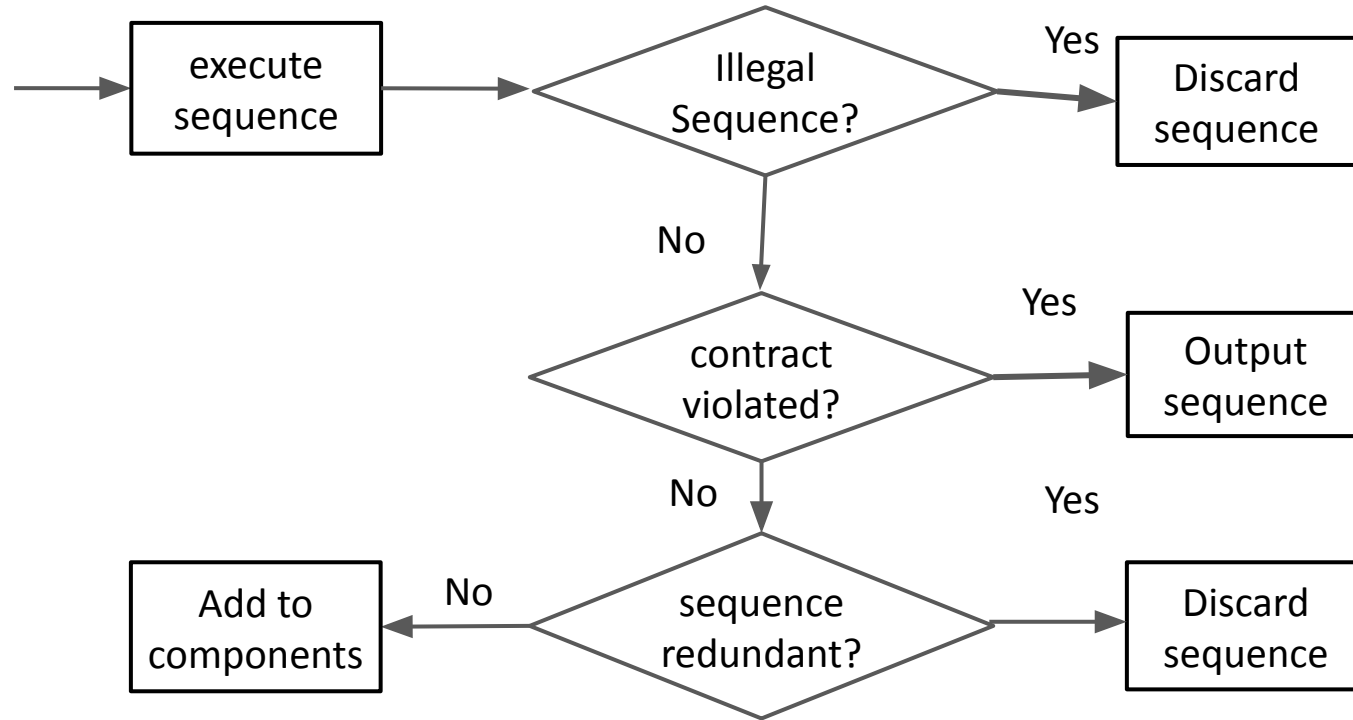
Randoop Algorithm

components = { int i = 0; boolean b = false; . . . }

Repeat until time limit expires:

- Create a new sequence
 - Randomly pick a method call $T_{\text{ret}} \ m(T_1, \dots, T_n)$
 - For each argument of type T_i , randomly pick sequence S_i from components that constructs an object v_i of that type
 - Create $S_{\text{new}} = S_1; \dots; S_n; \ T_{\text{ret}} \ v_{\text{new}} = m(v_1, \dots, v_n);$
- Classify new sequence S_{new} :
 - discard / output as test / add to components

Classifying a Sequence



Classifying a Sequence as Illegal

4. Illegal test

```
Date d = new Date(2006, 2, 14);  
d.setMonth(-1); // pre: argument >= 0  
assertTrue(d.equals(d));
```

5. Illegal test

```
Date d = new Date(2006, 2, 14);  
d.setMonth(-1);  
d.setDay(5);  
assertTrue(d.equals(d));
```

A **precondition** is a condition that must be true before method execution. A **precondition violation** usually leads to an exception

Classifying a Sequence as a Contract Violation

```
o.equals(o) == true  
o1.equals(o2) == o2.equals(o1)  
If (o1.equals(o2)) then o1.hashCode() == o2.hashCode()  
...
```

Classifying a Sequence as Redundant

- During generation, maintain a set of all objects created.
- A sequence is redundant if all the objects created during its execution are members of the above set (using `equals` to compare)

Exercise:

1. Write the smallest sequence Randoop can generate to create a valid `BinaryTree`
 - a. Hint: consider a default value for objects
2. Once generated, how does Randoop classify it?
 - a. Discard as illegal
 - b. Output as Contract Violating bug
 - c. Add to components for future extension
3. Extend the sequence you created in (1) to violate the assertion in `removeRoot()`. How would the sequence be classified?

```
class BinaryTree {  
    Node root;  
    public BinaryTree(Node r) {  
        root = r;  
    }  
    public Node removeRoot() {  
        assert(root != null);  
        ...  
    }  
}
```

```
class Node {  
    Node left;  
    Node right;  
    public Node(Node l, Node r) {  
        left = l; right = r;  
    }  
}
```

A Note on Assertions

- Major weakness of randoop!
- Can only find safety properties
- Also has a mode for regression oracles
 - How are regression oracles generated?

Generating Tests From a Different Perspective..

Search Based Software Testing

Idea: A test exists in a finite space of all possible Java tests. We just need to find the one which triggers the bug.

This requires two things

- 1) Prefix
- 2) Assertion

A note on this field...

So far we've discussed the **problem** of **test oracle generation** and the **tools** to solve it: **neural**, **neural constrained by grammar**, **NL parsing**, and **IR**.

Now we are going to discuss the **problem** of **search based software testing** and the tools to solve it: **feedback directed** (randoop), **genetic algorithms** (EvoSuite), and **coverage guided** (AFL).

Genetic Algorithms

- Search technique used to find true or approximate solutions to search or optimization problems
 - What are some optimization problems you know?
- Use techniques inspired by evolutionary biology:
 - Selection
 - Mutation
 - Crossover
- Motivating Example: Imagine you're trying to create a **meal plan** that meets daily nutritional requirements while **minimizing cost**. This is an **optimization problem** where we search for the best combination of foods.

Genetic Algorithms

Selection (Survival of the Fittest)

- **Biology:** In nature, organisms that are better adapted to their environment are more likely to survive and reproduce. This is known as **natural selection**.
- **Optimization:** In evolutionary algorithms, the best-performing candidate solutions (individuals) are selected from a population based on their "fitness" (how well they solve the problem). These individuals are more likely to contribute to the next generation.

Genetic Algorithms

Crossover (Genetic Recombination)

- **Biology:** During reproduction, genetic material from two parents combines to create offspring with traits inherited from both, leading to increased genetic diversity.
- **Optimization:** In evolutionary algorithms, crossover combines parts of two parent solutions to create a new solution (offspring) that may inherit beneficial traits from both parents, improving the search process.
- **Meal plan example - what would crossover look like? Breakfast from M1 lunch from M2**

Genetic Algorithms

Mutation (Genetic Variation)

- **Biology:** Mutations are random changes in an organism's DNA that introduce genetic diversity. While some mutations may be harmful, others can offer advantages that improve survival.
- **Optimization:** In evolutionary algorithms, mutation introduces small random changes to candidate solutions to maintain diversity and prevent the algorithm from getting stuck in local optima.
- **Meal plan example - what would mutation look like? Maybe swap out breakfast**

Genetic Algorithms

- Start with a population of randomly generated individuals
- Works in multiple “generations”
 - Select individuals for breeding based on “fitness”
 - Mutation + crossover creates individuals for the next generation
 - Used in the next iteration

Genetic Algorithm

Termination: The algorithm terminates when either:

- A max number of iterations have been reached or
- Timeout or
- A satisfactory fitness level has been achieved

NO CONVERGENCE GUARANTEES

Vocabulary

- **Individual** - Any possible solution **(also called chromosome)**
- **Population** - Group of all individuals
- **Fitness** – Target function that we are optimizing (each individual has a fitness)

Genetic Algorithms

To adapt GA to test generation (or any other task) we need to define the following:

1. What is a chromosome / individual in the population?
 - a. This is a potential solution
2. How are individuals selected for breeding?
3. What happens during crossover?
4. What happens during mutation?
5. How is fitness evaluated?
 - a. How do we measure how “good” a chromosome / individual is?
 - b. Called the *fitness function*

Meal Example

1. What is a chromosome?
 - a. Meal plan with one option for breakfast and one for dinner
2. How are individuals selected?
 - a. Probabilistically based on their fitness function
3. What happens during crossover?
 - a. The new individual has breakfast from one plan and lunch from another
4. What happens during mutation?
 - a. Breakfast is swapped out
5. Fitness function?
 - a. $F(\text{meal}) = \text{nutrition} - (0.5 \times \text{cost})$
 - b. $F(\text{mealplan}) = F(\text{breakfast}) + F(\text{lunch})$

Basic Genetic Algorithm

- Start with a large “population” of randomly generated “attempted solutions” to a problem
- Repeatedly do the following:
 - Evaluate each of the attempted solutions
 - (probabilistically) keep a subset of the best solutions
 - Use these solutions to generate a new population
- Quit when you have a satisfactory solution (or you run out of time)

Summary

Genetic Algorithms are search and optimization techniques inspired by natural selection and evolution.

Effective for complex, non-linear problems.

Explores a vast solution space efficiently.

Next week we will see how GAs are used to generate tests

Summary

- Randomized Testing
 - OOP: randomly select method to call. For arguments, use objects of that type that you have already created.
 - Problem: leads to redundant or illegal sequences
 - Solution: only keep legal and non-redundant sequences to extend